Spring MVC Architecture

1. What is Spring Web MVC?

Spring Web MVC, commonly called **Spring MVC**, is Spring's web framework used to build:

- Web applications
- REST APIs
- Backend services that handle HTTP requests

Officially, its module name is:

```
spring-webmvc
```

Spring MVC is built on top of the **Servlet API**. That means, even though we do not directly write `doGet()` or `doPost()` in Spring MVC, the application still works through servlet-based request handling internally.

---

2. What is Spring Boot's Role?

Spring Boot does **not** replace Spring MVC.

Spring Boot simply makes Spring MVC easier to use.

In a traditional Spring MVC application, we manually configure many things:

- Tomcat setup
- Spring application context
- DispatcherServlet registration
- DispatcherServlet mapping
- Component scanning
- JSON conversion

- Spring MVC configuration

Spring Boot does most of this automatically.

In a Spring Boot CRUD project, we usually write methods like:

```
@GetMapping("/{id}")
public Student getStudent(@PathVariable Integer id) {
    return studentService.getStudent(id);
}
```

We do not write:

```
doGet()
doPost()
doPut()
doDelete()
```

But internally, someone still has to receive the HTTP request from Tomcat, understand the URL and HTTP method, and then call the correct controller method.

That internal web system is **Spring MVC**.

The central servlet inside Spring MVC is called:

```
DispatcherServlet
```

---

3. DispatcherServletWhat is DispatcherServlet?

`DispatcherServlet` is the **front controller** of Spring MVC.

It receives incoming HTTP requests and coordinates the complete request-processing flow.

A simple mental model:

```
Client
  ↓
Tomcat
  ↓
DispatcherServlet
  ↓
Controller Method
  ↓
Response
```

Why Do We Need DispatcherServlet?

In a pure Servlet application, we may create different servlets for different URLs.

For example:

```
/student/create  → CreateStudentServlet
/student/get     → GetStudentServlet
/student/update  → UpdateStudentServlet
/student/delete  → DeleteStudentServlet
```

Each servlet may handle its own:

- URL matching
- Request parsing
- Response writing
- JSON conversion
- Error handling

This becomes repetitive as the project grows.

Spring MVC solves this by using one central servlet:

```
All requests first come to DispatcherServlet.
DispatcherServlet then forwards the request to the correct controller method.
```

That is why it is called **DispatcherServlet**.

It dispatches the request to the correct handler.

DispatcherServlet is Still a Servlet

DispatcherServlet is not magic.

It is also a servlet.

Tomcat knows how to call servlets. So when a request comes, Tomcat sends the request to DispatcherServlet.

Then DispatcherServlet takes help from other Spring MVC components to process the request.

4. High-Level Spring MVC Flow

```
Client sends HTTP request
        ↓
Tomcat receives request
        ↓
Tomcat sends request to DispatcherServlet
        ↓
DispatcherServlet asks HandlerMapping:
"Which controller method should handle this request?"
        ↓
HandlerMapping returns the matching controller method
        ↓
DispatcherServlet uses HandlerAdapter to call that method
        ↓
Request data is assigned to method parameters
        ↓
Controller calls Service
        ↓
Service calls Repository
        ↓
Response is returned
        ↓
Spring converts Java object to JSON
        ↓
Client receives response
```

DispatcherServlet is the coordinator.

It does not do everything by itself. It takes help from multiple Spring MVC components.

Important components:

Component	Responsibility
`DispatcherServlet`	Central servlet that receives and coordinates requests
`HandlerMapping`	Finds which controller method should handle the request
`HandlerAdapter`	Actually invokes the selected controller method
`HandlerMethodArgumentResolver`	Resolves method parameters like path variables, query parameters, request body
`HttpMessageConverter`	Converts JSON to Java object and Java object to JSON
`Jackson`	Common JSON library used for conversion

5. HandlerMapping

Why HandlerMapping is Needed

DispatcherServlet receives the request, but it does not directly know which controller method to call.

For example, if this request comes:

```
GET /students/1
```

DispatcherServlet needs to know that this should call:

```
@GetMapping("/{id}")
public Student getStudent(@PathVariable Integer id) {
    return studentService.getStudent(id);
}
```

This matching is done by **HandlerMapping**.

What is HandlerMapping?

HandlerMapping is the Spring MVC component that maps an HTTP request to the correct controller method.

It checks things like:

- URL path
- HTTP method
- Controller annotations
- Method-level mappings

Example:

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping("/{id}")
    public Student getStudent(@PathVariable Integer id) {
        return studentService.getStudent(id);
    }

    @PostMapping
    public Student createStudent(@RequestBody Student student) {
        return studentService.createStudent(student);
    }
}
```

Spring internally understands:

```
GET /students/1       → getStudent()
POST /students        → createStudent()
```

Even if two URLs look similar, the HTTP method can change the meaning.

For example:

```
GET /students     → fetch students
```

```
POST /students    → create student
```


When Does HandlerMapping Build This Mapping?

When the Spring MVC application starts:

1. `@ComponentScan` finds controller classes.
2. Spring creates controller objects as beans.
3. HandlerMapping reads annotations like:
    - `@RequestMapping`
    - `@GetMapping`
    - `@PostMapping`
    - `@PutMapping`
    - `@DeleteMapping`
4. It builds an internal mapping table.

Example internal mapping:

```
GET /students/{id}     → StudentController.getStudent()
POST /students         → StudentController.createStudent()
PUT /students/{id}     → StudentController.updateStudent()
DELETE /students/{id}  → StudentController.deleteStudent()
```

At runtime, HandlerMapping uses this prepared mapping table to quickly find the correct controller method.

6. HandlerAdapter

HandlerMapping only finds the correct method.

It does not execute the method.

The actual method invocation is done by another Spring MVC component called:

```
HandlerAdapter
```

Simple flow:

```
DispatcherServlet asks HandlerMapping:
"Which method should handle this request?"

HandlerMapping replies:
"StudentController.getStudent()"

DispatcherServlet then asks HandlerAdapter:
"Please call this method properly."
```

HandlerAdapter then prepares method arguments and invokes the controller method.

---

7. How Request Data Reaches the Controller Method

In a web API, request data usually comes from three places:

1. Path variable
2. Query parameter
3. Request body

Spring MVC uses argument resolvers to extract this data and assign it to controller method parameters.

7.1 Path Variable

Example request:

```
GET /students/10
```

Controller method:

```
@GetMapping("/{id}")
public Student getStudent(@PathVariable Integer id) {
    return studentService.getStudent(id);
}
```

Spring compares:

```
Request URL: /students/10
Endpoint:    /students/{id}
```

Then it assigns:

```
id = 10
```

This is handled by Spring MVC's argument resolution mechanism.

7.2 Query Parameter

Example request:

```
GET /students?name=Rohit
```

Controller method:

```
@GetMapping
public List<Student> getStudents(@RequestParam String name) {
    return studentService.getStudentsByName(name);
}
```

Here Spring extracts:

```
name = Rohit
```

Tomcat already parses the HTTP request and makes query parameters available through the servlet request object.

Spring MVC then reads those values and assigns them to controller method parameters.

7.3 Request Body

Example POST request:

```
POST /students
Content-Type: application/json
```

Request body:

```
{
  "name": "Rohit",
  "email": "rohit@example.com",
  "mobile": "9999999999"
}
```

Controller method:

```
@PostMapping
public Student createStudent(@RequestBody Student student) {
    return studentService.createStudent(student);
}
```

Here Spring MVC reads the JSON body and converts it into a Java object:

```
Student student
```

This is mainly done using:

```
HttpMessageConverter + Jackson
```

---

8. Who Converts JSON to Java Object?

In most Spring MVC and Spring Boot applications, JSON conversion is handled by **Jackson**.

Spring MVC uses `HttpMessageConverter` to convert:

```
JSON → Java Object
Java Object → JSON
```

Example:

```
{
  "name": "Rohit",
  "email": "rohit@example.com"
}
```

gets converted into:

```
Student student
```

And when a controller returns:

```
return student;
```

Spring converts that Java object back into JSON response.

---

9. Application Goal

We will create a small Student REST API.

The API will support:

```
POST   /students        → create student
GET    /students/{id}   → get student by id
GET    /students        → get all students
PUT    /students/{id}   → update student
DELETE /students/{id}   → delete student
```

Data will be stored in memory using:

```
Map<Integer, Student>
```

Layered flow:

```
Controller → Service → Repository
```

---

10. Project Structure

```
src
 └── main
     └── java
         └── in
             └── strikes
                 ├── App.java
                 ├── config
                 │   └── WebConfig.java
                 ├── controller
                 │   └── StudentController.java
                 ├── model
                 │   └── Student.java
                 ├── repository
                 │   └── StudentRepository.java
                 └── service
                     └── StudentService.java
```

---

11. pom.xml

Because we are not using Spring Boot, we manually add the required dependencies.

```
<dependencies>

    <!-- Spring MVC -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-webmvc</artifactId>
        <version>6.1.6</version>
    </dependency>

    <!-- Embedded Tomcat -->
    <dependency>
        <groupId>org.apache.tomcat.embed</groupId>
        <artifactId>tomcat-embed-core</artifactId>
        <version>10.1.20</version>
    </dependency>

    <!-- JSON conversion using Jackson -->
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
        <version>2.17.0</version>
    </dependency>

</dependencies>
```

Explanation of Dependencies1. `spring-webmvc`

This gives Spring MVC features such as:

- `DispatcherServlet`
- `@RestController`
- `@Controller`
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`
- HandlerMapping

- HandlerAdapter
- ViewResolver support

2. `tomcat-embed-core`

This allows us to start Tomcat directly from the Java `main()` method.

So instead of installing external Tomcat and deploying a WAR file, we can run:

```
tomcat.start();
```

This means Tomcat is embedded inside our Java application.

---

3. `jackson-databind`

This dependency helps convert:

```
JSON → Java Object
Java Object → JSON
```

Spring MVC uses `HttpMessageConverter`, and Jackson is commonly used behind the scenes for JSON conversion.

---

12. Student.java

```
package in.strikes.model;

public class Student {

    private Integer id;
    private String name;
    private String email;
    private String mobile;

    public Student() {
    }

    public Student(Integer id, String name, String email, String mobile) {
```

```
        this.id = id;
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getMobile() {
        return mobile;
    }

    public void setMobile(String mobile) {
        this.mobile = mobile;
    }
}
```

Explanation

This is a simple model class.

It represents one student.

A student has:

- `id`

- `name`
- `email`
- `mobile`

The no-argument constructor is important because Jackson needs it while converting JSON into a Java object.

---

13. StudentRepository.java

```
package in.strikes.repository;

import in.strikes.model.Student;
import org.springframework.stereotype.Repository;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

@Repository
public class StudentRepository {

    private final Map<Integer, Student> students = new HashMap<>();
    private Integer currentId = 1;

    public Student save(Student student) {
        student.setId(currentId);
        students.put(currentId, student);
        currentId++;
        return student;
    }

    public Student findById(Integer id) {
        return students.get(id);
    }

    public Collection<Student> findAll() {
        return students.values();
    }

    public Student update(Integer id, Student student) {
        Student existingStudent = students.get(id);

        if (existingStudent == null) {
            return null;
        }
```

```
        existingStudent.setName(student.getName());
        existingStudent.setEmail(student.getEmail());
        existingStudent.setMobile(student.getMobile());

        return existingStudent;
    }

    public Student delete(Integer id) {
        return students.remove(id);
    }
}
```

Explanation of `@Repository`

```
@Repository
```

This tells Spring:

```
This class belongs to the repository layer.
Please create and manage its object inside the IoC container.
```

Conceptually, this is the database layer.

Here, we are not using MySQL.

We are using:

```
Map<Integer, Student>
```

as an in-memory database.

In a real project, this repository would talk to the database using:

- JDBC
- JPA
- Hibernate
- Spring Data JPA

14. StudentService.java

```
package in.strikes.service;

import in.strikes.model.Student;
import in.strikes.repository.StudentRepository;
import org.springframework.stereotype.Service;

import java.util.Collection;

@Service
public class StudentService {

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    public Student createStudent(Student student) {
        return studentRepository.save(student);
    }

    public Student getStudent(Integer id) {
        return studentRepository.findById(id);
    }

    public Collection<Student> getAllStudents() {
        return studentRepository.findAll();
    }

    public Student updateStudent(Integer id, Student student) {
        return studentRepository.update(id, student);
    }

    public Student deleteStudent(Integer id) {
        return studentRepository.delete(id);
    }
}
```

Explanation of `@Service`

```
@Service
```

This tells Spring:

```
This class belongs to the service layer.
Please create and manage its object inside the IoC container.
```

The service layer contains business logic.

Right now, the logic is simple, but in real applications this layer may contain:

- Validation
- Business rules
- Calculations
- Calls to multiple repositories
- Transaction handling

Constructor Injection

```
public StudentService(StudentRepository studentRepository) {
    this.studentRepository = studentRepository;
}
```

This is dependency injection.

The service needs the repository, but it does not create the repository object manually.

We do not write:

```
StudentRepository studentRepository = new StudentRepository();
```

Instead, we ask Spring to provide the object.

This keeps the code loosely coupled and easier to test.

15. StudentController.java

```
package in.strikes.controller;

import in.strikes.model.Student;
import in.strikes.service.StudentService;
import org.springframework.web.bind.annotation.*;

import java.util.Collection;

@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentService studentService;

    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @PostMapping
    public Student createStudent(@RequestBody Student student) {
        return studentService.createStudent(student);
    }

    @GetMapping("/{id}")
    public Student getStudent(@PathVariable Integer id) {
        return studentService.getStudent(id);
    }

    @GetMapping
    public Collection<Student> getAllStudents() {
        return studentService.getAllStudents();
    }

    @PutMapping("/{id}")
    public Student updateStudent(@PathVariable Integer id,
                                 @RequestBody Student student) {
        return studentService.updateStudent(id, student);
    }

    @DeleteMapping("/{id}")
    public Student deleteStudent(@PathVariable Integer id) {
        return studentService.deleteStudent(id);
    }
}
```

Explanation of `@RestController`

```
@RestController
```

This tells Spring:

```
This class will handle REST API requests.
The return value of its methods should be written directly in the HTTP response body.
```

So when this method returns a `Student` object:

```
return student;
```

Spring converts it into JSON.

`@RestController` is basically a shortcut for:

```
@Controller + @ResponseBody
```

Explanation of `@RequestMapping`

```
@RequestMapping("/students")
```

This defines the base URL for this controller.

So every API inside this class starts with:

```
/students
```

Explanation of `@PostMapping`

```
@PostMapping
public Student createStudent(@RequestBody Student student) {
    return studentService.createStudent(student);
}
```

This handles:

```
POST /students
```

It is used to create a new student.

Explanation of `@RequestBody`

```
@RequestBody Student student
```

This means:

```
Read the JSON request body and convert it into a Student object.
```

Example request body:

```
{
  "name": "Rohit",
  "email": "rohit@example.com",
  "mobile": "9999999999"
}
```

Spring MVC uses Jackson through `HttpMessageConverter` to convert this JSON into:

```
Student student
```

Explanation of `@GetMapping("/{id}")`

```
@GetMapping("/{id}")
public Student getStudent(@PathVariable Integer id) {
    return studentService.getStudent(id);
}
```

This handles:

```
GET /students/1
```

Explanation of `@PathVariable`

```
@PathVariable Integer id
```

This means:

```
Take the value from the URL path and assign it to this variable.
```

Example:

```
/students/1
```

Here:

```
id = 1
```

16. WebConfig.java

```
package in.strikes.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;

@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "in.strikes")
public class WebConfig {
}
```

This file is very important because we are not using Spring Boot.

So we must manually tell Spring MVC what to do.

Explanation of `@Configuration`

```
@Configuration
```

This tells Spring:

```
This class contains Spring configuration.
```

Explanation of `@EnableWebMvc`

```
@EnableWebMvc
```

This enables Spring MVC features.

Without this, Spring MVC annotations may not work fully, such as:

- `@RestController`
- `@Controller`
- `@RequestMapping`

- `@GetMapping`
- `@PostMapping`
- `@RequestBody`
- `@PathVariable`

It also enables Spring MVC infrastructure such as:

- HandlerMapping
- HandlerAdapter
- Message converters
- Controller handling

Explanation of `@ComponentScan`

```
@ComponentScan(basePackages = "in.strikes")
```

This tells Spring:

```
Scan this package and find Spring-managed classes.
```

So Spring finds:

- `StudentController`
- `StudentService`
- `StudentRepository`

and manages them inside the IoC container.

---

17. App.java

This is the main class where we manually start embedded Tomcat and register Spring MVC.

```
package in.strikes;

import in.strikes.config.WebConfig;
import org.apache.catalina.Context;
import org.apache.catalina.startup.Tomcat;
import org.springframework.web.context.support.AnnotationConfigWebApplicationContext;
import org.springframework.web.servlet.DispatcherServlet;

import java.io.File;

public class App {

    public static void main(String[] args) throws Exception {

        Tomcat tomcat = new Tomcat();

        tomcat.setPort(8080);
        tomcat.getConnector();

        String contextPath = "";
        String docBase = new File("src/main/webapp").getAbsolutePath();

        Context context = tomcat.addContext(contextPath, docBase);

        AnnotationConfigWebApplicationContext springContext =
                new AnnotationConfigWebApplicationContext();

        springContext.register(WebConfig.class);

        DispatcherServlet dispatcherServlet =
                new DispatcherServlet(springContext);

        Tomcat.addServlet(context, "dispatcherServlet", dispatcherServlet);

        context.addServletMappingDecoded("/", "dispatcherServlet");

        tomcat.start();
        tomcat.getServer().await();
    }
}
```

18. App.java Step-by-Step Explanation

Step 1: Create Tomcat

```
Tomcat tomcat = new Tomcat();
```

We are manually creating an embedded Tomcat server.

In external Tomcat, Tomcat already exists outside the application.

Here, Tomcat is created inside our Java application.

Step 2: Set Port

```
tomcat.setPort(8080);
```

This means our server will run on:

```
http://localhost:8080
```

Step 3: Create Connector

```
tomcat.getConnector();
```

This initializes the connector that listens for HTTP requests.

Step 4: Create Web Context

```
String contextPath = "";
String docBase = new File("src/main/webapp").getAbsolutePath();

Context context = tomcat.addContext(contextPath, docBase);
```

This creates a web application context.

`contextPath` means the base path of the application.

For example, in external Tomcat, if we deploy:

```
student-api.war
```

then the context path usually becomes:

```
/student-api
```

So the URL becomes:

```
http://localhost:8080/student-api/students
```

But here:

```
String contextPath = "";
```

So our API runs directly at:

```
http://localhost:8080/students
```

not:

```
http://localhost:8080/app-name/students
```

What is `docBase`?

`docBase` is the physical folder representing the web application.

Tomcat expects a folder for the web application context.

In our REST API, we may not need static files, but Tomcat still expects a valid folder.

So we pass:

```
src/main/webapp
```

as the `docBase`.

Step 5: Create Spring Web Context

```
AnnotationConfigWebApplicationContext springContext =
        new AnnotationConfigWebApplicationContext();
```

This creates a Spring application context for a web application.

This is where Spring manages:

- Controllers
- Services
- Repositories
- Configuration classes

Step 6: Register Config Class

```
springContext.register(WebConfig.class);
```

This tells Spring:

```
Use WebConfig as the configuration class.
```

From `WebConfig`, Spring knows:

- Spring MVC should be enabled
- Which package should be scanned

- Which classes should become beans

Step 7: Create DispatcherServlet

```
DispatcherServlet dispatcherServlet =
        new DispatcherServlet(springContext);
```

This creates the central servlet of Spring MVC.

We pass `springContext` to DispatcherServlet because DispatcherServlet needs access to the Spring IoC container.

Why DispatcherServlet Needs Spring Context

DispatcherServlet receives the HTTP request from Tomcat.

But after receiving the request, it needs to know:

- Which controllers exist
- Which URL maps to which method
- Which services are available
- Which repositories are available
- Which message converters are available

All this information is inside the Spring IoC container.

That is why DispatcherServlet needs the Spring context.

Step 8: Register DispatcherServlet with Tomcat

```
Tomcat.addServlet(context, "dispatcherServlet", dispatcherServlet);
```

This tells Tomcat:

```
This web application has a servlet named dispatcherServlet.
```

Step 9: Map DispatcherServlet

```
context.addServletMappingDecoded("/", "dispatcherServlet");
```

This tells Tomcat:

```
Send all requests to DispatcherServlet.
```

So when a request comes:

```
GET /students/1
```

Tomcat sends it to DispatcherServlet.

Then DispatcherServlet finds and calls the correct controller method.

Step 10: Start Server

```
tomcat.start();
tomcat.getServer().await();
```

`tomcat.start()` starts the server.

`await()` keeps the server running.

Without `await()`, the `main()` method may finish and the application may stop immediately.

19. How to Run

Run the `App.java` file from the IDE.

Then test APIs using Postman or browser.

Example URLs:

```
POST http://localhost:8080/students
GET  http://localhost:8080/students
GET  http://localhost:8080/students/1
PUT  http://localhost:8080/students/1
DELETE http://localhost:8080/students/1
```

---

20. What Spring Boot Auto-Configures

In the manual Spring MVC application, we configured many things ourselves:

- Added Spring MVC dependency
- Added embedded Tomcat dependency
- Added Jackson dependency
- Created Tomcat manually
- Created Spring web context manually
- Registered WebConfig manually
- Created DispatcherServlet manually
- Registered DispatcherServlet with Tomcat
- Mapped DispatcherServlet to `/`
- Started Tomcat manually

In Spring Boot, most of this is automatic.

Spring Boot web applications include an embedded web server by default when we use:

```
spring-boot-starter-web
```

For servlet-based applications, Spring Boot brings embedded Tomcat by default.

21. Same App in Spring Boot

With Spring Boot, the `pom.xml` mainly needs:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

And the main class becomes:

```
@SpringBootApplication
public class StudentApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentApplication.class, args);
    }
}
```

That is it.

The controller, service, repository, and model can remain almost the same.

22. Manual Spring MVC vs Spring Boot

Work	Manual Spring MVC Embedded Tomcat	Spring Boot
Add Spring MVC dependency	Manually add `spring-webmvc`	Included in starter
Add embedded Tomcat	Manually add `tomcat-embed-core`	Included in starter
Add Jackson	Manually add `jackson-databind`	Included in starter
Start Tomcat	Manually in `App.java`	Automatic

Create Spring web context	Manual	Automatic
Register DispatcherServlet	Manual	Automatic
Map DispatcherServlet	Manual	Automatic
Enable Spring MVC	`@EnableWebMvc`	Auto-configured
Component scanning	`@ComponentScan`	Via `@SpringBootApplication`
Run application	Custom Tomcat code	`SpringApplication.run()`

23. `@Controller` vs `@RestController`

`@Controller`

`@Controller` is traditionally used when we want to return web pages.

Example:

```
@Controller
public class HomeController {

    @GetMapping("/home")
    public String home() {
        return "home";
    }
}
```

Here:

```
return "home";
```

does not mean response body `"home"`.

It means:

```
Find and return the home view page.
```

That view page could be:

```
home.jsp
```

`@RestController`

`@RestController` is used for REST APIs.

Example:

```
@RestController
public class StudentController {

    @GetMapping("/student")
    public Student getStudent() {
        return new Student(1, "Rohit", "rohit@example.com", "9999999999");
    }
}
```

Here the return value means:

```
Convert this Java object into JSON and send it in the HTTP response body.
```

So:

```
return student;
```

becomes:

```
{
  "id": 1,
  "name": "Rohit",
  "email": "rohit@example.com",
```

```
  "mobile": "9999999999"
}
```

Simple difference:

```
@Controller      → usually returns views/pages
@RestController  → usually returns data/JSON
```

---

24. Why ViewResolver Exists

Spring MVC was not created only for REST APIs.

Originally, Spring MVC was also heavily used to create full web applications where the backend returned HTML pages.

In that style, the controller does not return JSON.

The controller returns the name of a view page.

Example:

```
@GetMapping("/home")
public String home() {
    return "home";
}
```

Here:

```
return "home";
```

means:

```
Find the home page and render it.
```

That page could be:

```
/WEB-INF/views/home.jsp
```

This job is done by **ViewResolver**.

---

25. ViewResolver Example

If the controller returns:

```
return "home";
```

and ViewResolver is configured like this:

```
resolver.setPrefix("/WEB-INF/views/");
resolver.setSuffix(".jsp");
```

then Spring converts:

```
home
```

into:

```
/WEB-INF/views/home.jsp
```

So ViewResolver maps logical view names to actual view files.

---

26. JSP MVC Project Structure

```
src
 └── main
     ├── java
     │   └── in
     │       └── strikes
```

```
│           ├── App.java
│           ├── config
│           │   └── WebConfig.java
│           └── controller
│               └── HomeController.java
│
└── webapp
    ├── WEB-INF
    │   └── views
    │       └── home.jsp
    └── resources
        └── css
            └── style.css
```

Important:

```
JSP files are kept inside WEB-INF so users cannot directly access them from the browser.
```

The request should go through the controller first.

---

27. Extra Dependency for JSP

For JSP support, we need Tomcat's JSP engine.

Add:

```
<dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
    <version>10.1.20</version>
</dependency>
```

`tomcat-embed-jasper` is needed because JSP pages require Tomcat's JSP engine, called Jasper.

---

28. WebConfig.java for JSP

```
package in.strikes.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.ViewResolver;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.web.servlet.view.InternalResourceViewResolver;

@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "in.strikes")
public class WebConfig implements WebMvcConfigurer {

    @Bean
    public ViewResolver viewResolver() {
        InternalResourceViewResolver resolver =
                new InternalResourceViewResolver();

        resolver.setPrefix("/WEB-INF/views/");
        resolver.setSuffix(".jsp");

        return resolver;
    }

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/resources/**")
                .addResourceLocations("/resources/");
    }
}
```

Explanation of ViewResolver

```
resolver.setPrefix("/WEB-INF/views/");
resolver.setSuffix(".jsp");
```

When controller returns:

```
return "home";
```

ViewResolver converts it into:

```
/WEB-INF/views/home.jsp
```

So:

```
home → /WEB-INF/views/home.jsp
```

Explanation of `addResourceHandlers`

```
registry.addResourceHandler("/resources/**")
        .addResourceLocations("/resources/");
```

This allows the browser to access static files like CSS.

For example:

```
/resources/css/style.css
```

will load:

```
src/main/webapp/resources/css/style.css
```

Without this configuration, CSS may not load properly because with `@EnableWebMvc`, we are manually controlling Spring MVC configuration.

---

29. HomeController.java

```
package in.strikes.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
```

```
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/home")
    public String home(Model model) {

        model.addAttribute("title", "Spring MVC");
        model.addAttribute("message", "Welcome to Spring MVC with JSP");

        return "home";
    }
}
```

Explanation of `@Controller`

```
@Controller
```

This is used when we want to return a view page like JSP.

Explanation of Model

```
Model model
```

Model is used to send data from the controller to the JSP page.

Simple MVC meaning:

```
Model      → data
View       → JSP page
Controller → handles request and prepares model
```

Example:

```
model.addAttribute("title", "Spring MVC");
```

```
model.addAttribute("message", "Welcome to Spring MVC with JSP");
```

These values can be used inside JSP.

30. home.jsp

Create this file:

```
src/main/webapp/WEB-INF/views/home.jsp
```

Example:

```
<!DOCTYPE html>
<html>
<head>
    <title>${title}</title>
    <link rel="stylesheet" href="/resources/css/style.css">
</head>
<body>
    <h1>${title}</h1>
    <p>${message}</p>
</body>
</html>
```

Explanation

JSP is the View.

It receives data from the Model.

These values:

```
${title}
${message}
```

come from the controller:

```
model.addAttribute("title", "Spring MVC");
model.addAttribute("message", "Welcome to Spring MVC with JSP");
```

---

31. style.css

Create this file:

```
src/main/webapp/resources/css/style.css
```

Example:

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f4;
    padding: 40px;
}

h1 {
    color: #222;
}

p {
    font-size: 18px;
}
```

---

32. Important Difference Between REST API and JSP Example

In the REST API example, we mainly returned JSON.

So the focus was:

```
Controller → Service → Repository → JSON Response
```

But in the JSP example, we return a view page.

So the flow becomes:

```
Controller → Model → ViewResolver → JSP Page
```

Earlier, the REST API did not really need JSP files or static web pages.

But in the JSP example, we need:

```
src/main/webapp
```

because it contains:

- JSP files
- CSS files
- Static resources

Here, `docBase` becomes more important because Tomcat needs to find the actual web application files.

---

33. Final MVC Meaning

Spring MVC stands for:

```
Model - View - Controller
```

In a web-page-based MVC application:

Part	Meaning
Model	Data sent from controller to view
View	JSP page or UI page
Controller	Handles request and decides response/view

Example:

```
User opens /home
        ↓
DispatcherServlet receives request
        ↓
HandlerMapping finds HomeController.home()
        ↓
Controller adds data to Model
        ↓
Controller returns "home"
        ↓
ViewResolver converts "home" to /WEB-INF/views/home.jsp
        ↓
JSP is rendered
        ↓
HTML response is sent to browser
```

---

34. Final Summary

Spring MVC is the web framework that handles HTTP requests in Spring applications.

The most important component is:

```
DispatcherServlet
```

It receives every request and coordinates with other Spring MVC components.

The main request flow is:

```
Tomcat
  ↓
DispatcherServlet
  ↓
HandlerMapping
  ↓
HandlerAdapter
  ↓
Controller
  ↓
Service
  ↓
Repository
```

```
  ↓
Response
```

For REST APIs:

```
Java Object → JSON
```

is handled using:

```
HttpMessageConverter + Jackson
```

For web pages:

```
View name → JSP page
```

is handled using:

```
ViewResolver
```

Spring Boot makes all of this easier by auto-configuring most of the setup.

But understanding manual Spring MVC helps us understand what Spring Boot is doing behind the scenes.